

Prueba Técnica — Desarrollador de Software Full Stack Junior

1. Objetivo

El candidato deberá desarrollar una aplicación web Full Stack para la gestión de solicitudes internas de soporte tecnológico.

La prueba busca evaluar conocimientos de desarrollo frontend y backend, manejo de bases de datos, diseño de APIs, lógica de negocio, validaciones, control de versiones y capacidad para tomar y justificar decisiones técnicas.

2. Contexto

Una organización requiere una aplicación que permita registrar y realizar seguimiento a solicitudes de soporte tecnológico.

Cada solicitud deberá permitir identificar:

- Título.
- Descripción.
- Usuario solicitante.
- Categoría.
- Prioridad.
- Estado.
- Fecha de creación.
- Fecha de actualización.

Las categorías disponibles serán:

- Hardware
- Software
- Red
- Accesos
- Otros

Las prioridades disponibles serán:

- Baja
- Media
- Alta
- Crítica

Los estados disponibles serán:

- Pendiente

- En progreso
 - Resuelta
 - Cancelada
-

3. Funcionalidades requeridas

La aplicación deberá permitir como mínimo:

- Crear solicitudes.
- Consultar solicitudes.
- Consultar el detalle de una solicitud.
- Editar solicitudes.
- Cambiar el estado de una solicitud.
- Eliminar o cancelar solicitudes.
- Consultar el historial de cambios de una solicitud.
- Buscar solicitudes.
- Filtrar por estado, prioridad y categoría.
- Ordenar los resultados.
- Pagar los resultados.

Cada cambio de estado deberá quedar registrado en el historial de la solicitud, indicando como mínimo:

- Estado anterior.
 - Estado nuevo.
 - Fecha y hora.
 - Usuario responsable.
 - Observación, cuando corresponda.
-

4. Reglas de negocio

La aplicación deberá contemplar reglas que permitan controlar las transiciones entre los diferentes estados de una solicitud.

Como mínimo:

- Una solicitud pendiente podrá pasar a en progreso.
- Una solicitud en progreso podrá pasar a resuelta.
- Una solicitud pendiente podrá ser cancelada.
- Una solicitud cancelada no podrá volver a estar en progreso.
- Una solicitud resuelta no podrá volver a estar en progreso.
- Una solicitud con prioridad crítica deberá registrar una observación al ser resuelta.

El candidato podrá implementar reglas adicionales si considera que aportan valor a la solución.

Las decisiones adicionales deberán estar justificadas en la documentación del proyecto.

5. Tecnologías

La solución podrá desarrollarse utilizando las tecnologías que el candidato considere apropiadas.

Backend

Se podrá utilizar cualquier lenguaje o framework.

Se recomienda:

- JavaScript.
- Node.js.
- Express.

Frontend

Se podrá utilizar cualquier framework o tecnología.

Se recomienda:

- Vue 3.

Base de datos

Se podrá utilizar una base de datos relacional o no relacional.

Se recomienda:

- PostgreSQL.
- MongoDB.

La selección de tecnologías deberá ser justificada por el candidato.

6. Arquitectura

El candidato deberá seleccionar y aplicar una arquitectura para organizar la solución.

La arquitectura será de libre elección.

La solución deberá incluir una breve justificación de:

- Arquitectura seleccionada.
- Organización del proyecto.
- Separación de responsabilidades.
- Principales decisiones técnicas.

No se exige una arquitectura específica.

7. Validaciones

La aplicación deberá implementar validaciones en:

Frontend

Los campos de los formularios deberán contar con las validaciones correspondientes.

Backend

El backend deberá implementar validaciones utilizando DTOs, Schemas, Validators o un mecanismo equivalente.

Las validaciones deberán contemplar como mínimo:

- Campos obligatorios.
 - Tipos de datos.
 - Valores permitidos.
 - Datos inválidos.
 - Reglas necesarias para las operaciones solicitadas.
-

8. Control de versiones

El uso de Git es **obligatorio**.

El proyecto deberá encontrarse en un repositorio disponible en una plataforma como:

- GitHub.
- GitLab.
- Bitbucket.
- Otra plataforma equivalente.

El repositorio deberá contener como mínimo:

- Rama `main`.
- Rama `develop`.
- Mínimo **5 commits**.
- Al menos una integración de `develop` hacia `main`.

Los commits deberán reflejar el proceso de desarrollo de la solución.

9. Documentación

El proyecto deberá incluir un `README.md` con la información necesaria para:

- Instalar el proyecto.

- Configurar las variables de entorno.
- Configurar la base de datos.
- Ejecutar frontend y backend.
- Ejecutar la aplicación.
- Conocer las principales decisiones técnicas.

También deberán documentarse los supuestos o decisiones tomadas cuando un requerimiento no se encuentre definido explícitamente.

10. Uso de Inteligencia Artificial

El uso de herramientas de Inteligencia Artificial está permitido durante el desarrollo de la prueba.

Estas herramientas podrán utilizarse como apoyo para:

- Consultar conceptos.
- Investigar alternativas.
- Generar código.
- Revisar código.
- Identificar errores.
- Resolver problemas puntuales.

Sin embargo, su utilización deberá realizarse **con moderación y como herramienta de apoyo al desarrollo**.

El candidato será responsable de la totalidad del código entregado y deberá comprender las soluciones implementadas.

Durante una eventual revisión técnica, podrá solicitarse al candidato explicar o modificar cualquier parte de la solución.

11. Funcionalidades opcionales

Las siguientes funcionalidades son opcionales y no son necesarias para completar la prueba:

Documentación de API

Implementación de documentación mediante:

- Swagger / OpenAPI.
- Otra tecnología equivalente.

Contenedorización

Implementación mediante:

- Docker.

- Docker Compose.
- Otra tecnología equivalente.

Pruebas automatizadas

Implementación de pruebas unitarias.

Se recomienda:

- Jest.
- Otra tecnología equivalente.

Sistema de notificaciones

Implementación de un mecanismo de notificación cuando ocurran determinados eventos de la aplicación.

La notificación podrá realizarse mediante:

- Correo electrónico.
- Notificaciones dentro del cliente.
- WebSockets.
- Webhooks.
- Eventos.
- Otra solución técnicamente justificable.

12. Entregables

El candidato deberá proporcionar:

1. URL del repositorio Git.
2. Instrucciones de ejecución mediante `README.md`.
3. Código fuente del frontend.
4. Código fuente del backend.
5. Configuración o scripts necesarios para la base de datos.
6. Archivo `.env.example` o equivalente.

13. Tiempo estimado

Tiempo recomendado:

4 a 6 horas.

Las funcionalidades opcionales no deberán implementarse en detrimento de los requerimientos obligatorios.

Se valorará más una solución correctamente implementada, estructurada y justificada que una solución con un mayor número de funcionalidades pero con deficiencias técnicas.

Consideración final

La prueba busca evaluar la capacidad del candidato para **analizar un problema, diseñar una solución, implementarla y justificar sus decisiones técnicas**, además de demostrar fundamentos suficientes para desempeñarse como desarrollador Full Stack Junior.